Lctr 13: Introduction to Real Numbers

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Floating-point real number representation.
  - Definition of floating-point numbers in C.
- Fixed-point real number representation.
  - Unsigned fixed-point representation.
  - Signed fixed-point representation.
  - Definition of fixed-point numbers in C.
- Mathematical operations using fixed-point numbers.
  - Fixed-point addition and subtraction.
  - Fixed-point multiplication.

Demo project:

- Real number operations based on fixed-point integers.

Additional reading:

- Including DSP (digital signal processing) library in a project in CubeIDE.
- Using FPU (floating-point unit) and floating-point functions.

13.1 Introduction

13.2 Floating-point real number representation

13.2.1 Number representation in IEEE 754 standard

To be able to exchange the floating-point real numbers in different systems (programs), we have to follow a certain standard, such as the IEEE 754 standard (https://en.wikipedia.org/wiki/IEEE_754), which is the most widely used standard for floating-point numbers. The most commonly used IEEE 754-complient floating-point numbers are expressed in 4 and 8 bytes. They are called **single precision** and **double precision**, respectively.

13.2.1.1 The normal representation

The **normal representation** of an IEEE 754-compliantt floating-point real number, f, is given below:

$$f_N = (-1)^S(1 + F_R)2^{E_B},$$

where

- S is the sign of the number. It is 0 for a positive number and 1 for negative.
- F_R is the fraction of the number, which is confined to be $0 \leq F_R < 1$. (We can think of subscript R stands for real number.)
- E_B is the true exponent, which is a signed value. (We can think of subscript B stands for bias, as will be detailed later.)

13.2.1.2 The extended representations

In general, we have the following extended representations, including special numbers, according to the **definitions** in the standard:

$$f = \begin{cases} +0, & \text{if } S = 0, E = 0, F = 0, \\ -0, & \text{if } S = 1, E = 0, F = 0, \\ +\infty, & \text{if } S = 0, E = 2B + 1, F = 0, \\ -\infty, & \text{if } S = 1, E = 2B + 1, F = 0, \\ \text{NaN}, & \text{if } E = 2B + 1, F > 0, \\ f_N, & \text{if } 0 < E < 2B + 1, \\ f_S, & \text{if } E = 0, F > 0. \end{cases}$$

These representations can be classified into the following:

- Two zeros.
- Positive and negative infinities.
- NaN (not a number).
- f_N, the *normal* representation, which is the one we use most.
- f_S, the subnormal representation, which will be discussed next.

13.2.1.3 The subnormal representation

Note that the normal numbers that are closest to zero(s) in single precision are the following, as E can be as small as 1:

$$(-1)^S \times (1 + 0) \times 2^{1-127} = (-1)^S \times 2^{-126} \approx (-1)^S \times 1.18 \times 10^{-38}.$$

To express a real number that is closer to zero than the above two (minimum) normal numbers, we can use the **subnormal** representation, given below.

$$f_S = (-1)^S \times F_R \times 2^{-126}.$$

13.2.2 Using floating-point numbers in C

- To use floating-point numbers in C, we need to define them, as shown below.

```
float pi = 3.1415926;
```

- To convert an integer to a floating-point number, we just need to type-cast it. Note that if we have a floating-point number in a statement, the results will be automatically *casted* to a floating number (implicitly). For example, the printout of the following code

```
int a = 123;
float f = (float)a;
printf("Values of f and f/a are %f and %f, respectively.\n
    ", f, f/a);
```

should be

```
Values of f and f/a are 123.000000 and 1.000000,
    respectively.
```

- To convert a floating-point number to an integer, we can use the `lrint` function to round the floating to the nearest integer. (Here, `l` stands for `long int`; `rint` stands for *round to integer*. If we want to round to 64-bit integer, we need to use `llrint`.) For example, the printout of the following code

```
float f1 = 123.45;
float f2 = 123.54;
int p1 = lrint(f1);  // should be 123
int p2 = lrint(f2);  // should be 124
int m1 = lrint(-f1); // should be -123
int m2 = lrint(-f2); // should be -124
printf("Values of p1 and p2 are %d and %d, respectively.\n
    ", p1, p2);
printf("Values of m1 and m2 are %d and %d, respectively.\n
    ", m1, m2);
```

should be

```
Values of p1 and p2 are 123 and 124, respectively.
Values of m1 and m2 are -123 and -124, respectively.
```

13.3 Fixed-point real number representation

When we don't have an FPU (or even if we have one, but we don't want to have the overhead incurred when using the FPU), we can use fixed-point real numbers to express appropriate real numbers in a reduced-precision-but-improved-efficiency manner.

13.3.1 Unsigned fixed-point representation

13.3.1.1 Parts of unsigned fixed-point representation

The notation for unsigned fixed-point representation is UQm.n, where:

- U stands for unsigned.
- Q is reminiscent of the standard symbol for the set of rational numbers, which can be expressed as p/q, where both p and q are integers with q being nonzero.
- m is the number of bits used to express the whole-number (integer) part of the real number,
- n is the number of bits for the fraction part.

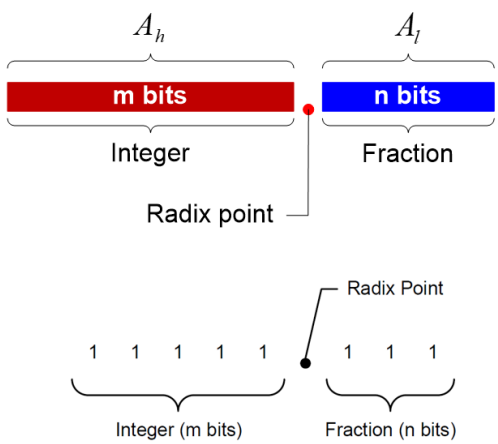

Figure 1: Illustration of UQm.n representation.

There are two approaches to decode the real number from an UQm.n encoded data shown in Figure 1. They are called integer and fraction approaches, respectively, according to their first step calculation.

13.3.1.2 Decoding an unsigned fixed-point data to obtain a real number using the integer approach

We can first express A_h and A_l together as an integer I_A by ignoring the radix point, which is fictional anyway. Then, we convert I_A to the real number.

Here, we have

$$I_A = A_h \times 2^n + A_l.$$

Then, we convert it to real number as

$$f_A = I_A \times 2^{-n}.$$

13.3.1.3 Decoding an unsigned fixed-point data to obtain a real number using the fraction approach

We can express the real number, f_A, directly using the format of radix point as

$$f_A = A_h + A_l \times 2^{-n}.$$

13.3.1.4 Examples for decoding UQm.n

While the fraction approach *may* be easier to calculate by hand, the integer approach is easier to use from programming's perspective, as it provides a good bridge to using the integer arithmetic operations.

Example 2. Consider an 8-bit UQ5.3 encoded data 0xFF (as shown in Figure 1). Decode its value to the real number.

Solution:

It is easy to see that $A_h = 31$ and $A_l = 7$. Using the fraction approach, we have

$$f_A = 31 + 7 \times 2^{-3} = 31 + 7/8 = 31.875.$$

Alternatively, using the integer approach, we have

$$f_A = \text{0xFF} \times 2^{-3} = 255/8 = 31.875.$$

> -:-

13.3.1.5 Encoding a real number to an unsigned fixed-point data

For encoding, we use exactly the reversed procedure of decoding with the integer approach. We just need to handle the case where we don't have enough bits to express the real number accurately by rounding (0.5+ goes up to 1, otherwise goes down to 0).

Example 4. Encode $f = 3.141593$ to UQ4.12 format.

Solution:

We follow the reversed procedure of the integer encoding approach:

- Calculate $f \times 2^{12} = 12867.964928$. (We use 2^{12} since there are 12 bits for the fraction part.)
- **Round** the result to the nearest integer: $\lfloor 12867.964928 \rceil$=12868.
- Convert the integer to binary: 12868 = 0b0011_0010_0100_0100.
- Convert to hexadecimal: 0x3244.

If we are interested in the rounding error, we have

$$|12868 - 12867.964928| * 2^{-12} = 8.56 \times 10^{-6}.$$

> -:-

13.3.2 Signed fixed-point representation

The notation for signed fixed-point representation is Qm.n. Similar to UQm.n, m and n are the number of bits of the whole-number and fraction parts, respectively, as shown in Figure 2. Note that there are altogether 1 + m + n bits for Qm.n coded data as **there is a sign bit not counted in m or n**.

Figure 2: Illustration of the Qm.n representation.

13.3.2.1 Decoding a Qm.n fixed-point data to a real number using the integer TC approach

There are many approaches for the decoding of a Qm.n fixed-point encoded data to a real number. Here we provide an easy and systematic approach, called integer TC (two's complement) approach, which is based on the decimal TC approach of the signed integers we discussed earlier.

Let $N = 1 + m + n$ be the total number of bits for a Qm.n encoded data, which is expressed as

$$0\text{b } b_{N-1}b_{N-2}...b_1b_0.$$

Then *the corresponding unsigned integer value* U_A can be written as

$$U_A = b_{N-1} \times 2^{N-1} + \sum_{n=0}^{N-2} b_n \times 2^n.$$

Now, we can convert it to *the corresponding signed integer value* I_A

$$\begin{aligned} I_A &= U_A - b_{N-1} \times 2^N \\ &= -b_{N-1} \times 2^{N-1} + \sum_{n=0}^{N-2} b_n \times 2^n \\ &= \begin{cases} U_A, & \text{if MSB } b_{N-1} = 0, \\ U_A - 2^N, & \text{otherwise.} \end{cases} \end{aligned}$$

With I_A, we can proceed to obtain the value of the real number by scaling I_A using 2^{-n}, as shown below

$$f = I_A \times 2^{-n}.$$

Example 5. Decode the real number represented by an 8-bit Q4.3 encoded data 0xAD using the integer TC approach.

Solution:

We follow the standard procedure of the integer-first TC approach below for the decoding:

- Find I_A from the encoded data 0xAD = 0b1010_1101, which is

$$I_A = -128 + 32 + 8 + 4 + 1 = -83.$$

- Multiply the coefficient $2^{-3} = 0.125$, we have $f_A = -10.375$.

Note that we can find I_A using the following steps as well:

- $U_A =$ `0xAD` = 173.
- MSB = 1 => $I_A = U_A - 256 = -83$.

> -:-

13.3.2.2 Decoding a Qm.n fixed-point data to a real number using the fraction TC approach

13.3.2.3 Encoding a real number to the Qm.n fixed-point format

We will use an example to illustrate how to do this using the reversed procedure of the integer TC approach for decoding.

Example 7. Encode f = -3.1234 in Q5.10 format.

Solution:

- Calculate $f \times 2^{10} = -3198.3616$.
- Round the result to the nearest integer: $\lfloor -3198.3616 \rceil = -3198$.
- Find the decimal TC as $-3198 + 2^{16} = 62338$
- Convert to binary and then hexadecimal as: 0b1111_0011_1000_0010 = 0xF382.

The error is $0.3616/2^{10} = 0.00035$.

> -:-

13.4 Arithmetic operations of fixed-point numbers

13.4.1 Fixed-point addition and subtraction

$$\begin{cases} I_A = f_A \times 2^{16} \\ I_B = f_B \times 2^{16} \\ I_C = f_C \times 2^{16} \end{cases} \Rightarrow \begin{cases} f_A = I_A \times 2^{-16} \\ f_B = I_B \times 2^{-16} \\ f_C = I_C \times 2^{-16} \end{cases}$$

$$\Rightarrow \quad \begin{aligned} f_C &= f_A + f_B \\ &= I_A \times 2^{-16} + I_B \times 2^{-16} \\ &= (I_A + I_B) \times 2^{-16} \end{aligned}$$

$$\Rightarrow \quad I_C \times 2^{-16} = (I_A + I_B) \times 2^{-16}$$

$$\Rightarrow \quad I_C = I_A + I_B$$

Figure 3: Addition of Q15.16 numbers.

While the addition and subtraction of fixed-point numbers are straightforward, multiplications should be handled differently **with shifting**, as shown below.

13.4.2 General idea for fixed-point multiplication

First, we use Q15.16 to illustrate the multiplication of fixed-point numbers, as shown in Figure 4.

$$\left.\begin{aligned} f_C &= f_A \times f_B \\ &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\ &= (I_A \times I_B) \times 2^{-32} \\ \\ f_C &= I_C \times 2^{-16} \end{aligned}\right\} \quad I_C = (I_A \times I_B) \times 2^{-16}$$

Figure 4: Multiplication of Q15.16 numbers.

For the above multiplication, we have the following notes:

- Both I_A and I_B are 32-bit numbers.
- $I_T = I_A \times I_B$ is a 64-bit number; subscript T means the temporary value.
- I_C is a 32-bit number. To obtain I_C from I_T, we need to shift I_T to right by 16 bits. Then, we need to type-cast it to 32 bits.
- For the Qm.n format, the number of bits to shift to the right is n.
- Before the multiplication, we need to scale the values of I_A and I_B to avoid the overflow. This is part of the algorithm design.

Then, we use some Q3.2 numbers to illustrate the above principle:

- $f_A = 2.5$ and $I_A = 10$ = 0b001010.
- $f_B = 2.0$ and $I_B = 8$ = 0b001000.
- $I_T = I_A \times I_B = 80$ = 0b0000_0101_0000.
- $I_C = I_T >> 2$ = 0b0101_00 ==> $f_C = 5.0$.

Of course, if f_B were 4.0, we would have an overflow with the multiplication, as shown below:

- $f_B = 4.0$ and $I_B = 16$ = 0b010000.
- $I_T = I_A \times I_B = 160$ = 0b0000_1010_0000.
- $I_C = I_T >> 2$ = 0b1010_00 = 40. Now, the MSB is 1 ==> $I_C = 40 - 64 = -24$ ==> $f_C = -6.0$.

13.4.3 Q15 and Q31 and definitions in C

Qm.n is the general format, and in real applications, the mostly used formats are Q15 and Q31, which are the shorthand for Q0.15 and Q0.31, respectively. The are used in signal processing, defined as `q15_t` and `q31_t`, respectively, derived from `int16_t` and `int32_t`, in `arm_math.h`.

Here are a number of notes for these data types:

- Their range for real numbers is from -1.0 to +1.0.
  - -1.0 is encoded as `0x8000` and `0x8000_0000`, respectively, in Q15 and Q31.
  - $+1.0$ cannot be expressed in these two formats. The maximum numbers are `0x7FFF` and `0x7FFF_FFFF`, respectively, which are just a bit short of 1.0.
- There can be an issue with $(-1.0) \times (-1.0)$. Mathematically, this should be 1.0. Yet, as we don't have 1.0 in the number system, the result will be -1.0 again. This can be seen below using integer multiplication for Q15:
  - $I_A = I_B = 0\text{x}8000 = 2^{15}$
  - $I_T = I_A \times I_B = 2^{30}$
  - $I_C = I_T \gg 15 = 2^{15}$, which corresponds to -1.0.

  To address this issue, we can scale -1.0 a bit to `0x8001`. The issue and solution are illustrated in the demo project below.

Example 8. Consider the following operations on Q15 numbers: (a) 0.25 + 0.5, (b) 0.25 - 0.5, and (c) 0.25 * 0.5. Perform the following:

1. Encode the two individual real numbers above, 0.25 and 0.5, by using the Q15 format.
2. Find the results of the addition, subtraction, and multiplication of the encoded data obtained in Part 1 (using the unsigned decimal TC approach discussed in Lctr 4 as needed).
3. Decode the results in Part 2 back to real numbers.

Solution:

Part 1.

(a) $0.25 \times 2^{15} = 8192$ = 0b0010_0000_0000_0000 = 0x2000. (Or, $0.25 \times 2^{15} = 0.25 \times 2^3 \times 2^{12} = 2 \times 2^{12}$ = 0x2000.)

(b) $0.5 \times 2^{15} = 16384$ = 0b0100_0000_0000_0000 = 0x4000. (Or, $0.5 \times 2^{15} = 0.5 \times 2^3 \times 2^{12} = 4 \times 2^{12}$ = 0x4000.)

Part 2.

(a) `0x2000+ 0x4000= 0x6000`.

(b) `0x2000- 0x4000= 0x2000+ (0x1_0000- 0x4000)= 0x2000+ 0xC000= 0xE000`.

Note that we have used TC to convert subtraction to addition here.

(c) $I_T = 2 \times 2^{12} \times 4 \times 2^{12} = 2^{27}$. So, we have $I_C = I_T >> 15 = 2^{12} =$ `0x1000`.

Part 3.

(a) 0.25 + 0.5 => 0x6000 $\times 2^{-15} = 6 \times 2^{12} \times 2^{-15} = 6 \times 2^{-3} = 0.75$.

(b) 0.25 - 0.5 => 0xE000 $\times 2^{-15} = 14 \times 2^{12} \times 2^{-15} = 14 \times 2^{-3} = 1.75$. The MSB is 1, so the value should be $1.75 - 2^{0+1} = -0.25$ when interpreted as a signed real number.

(c) $I_C \times 2^{-15} = 2^{-3} = 0.125$.

> -:-

13.5 Demo project

In the demo project, `ee1s_real_numbers`, we experiment with the `q31_t` and `q15_t` data types which are provided in the ARM DSP (digital signal processing) library.

This project experiments the following:

- The conversion between Q15 / Q31 and floating-point.
- Printout of the Q15 / Q31 with the help of the floating-point numbers.
- Illustration of the above-mentioned -1.0×-1.0 issue and the solution.

Additionally, we also program a function for Q31 multiplication, based on the General idea for fixed-point multiplication section we have just discussed above, as shown below.

```
// CMSIS DSP has arm_mult_q31 function. Here we write it directly
q31_t mp_mult_q31(q31_t x, q31_t y) {
    q63_t prod = (q63_t)x * y;
    return (q31_t) (prod >> 31);
}
```

See the Including DSP library in a project in CubeIDE section below for details about how to include DSP library in the project.

13.6 Additional reading

13.6.1 Concept map

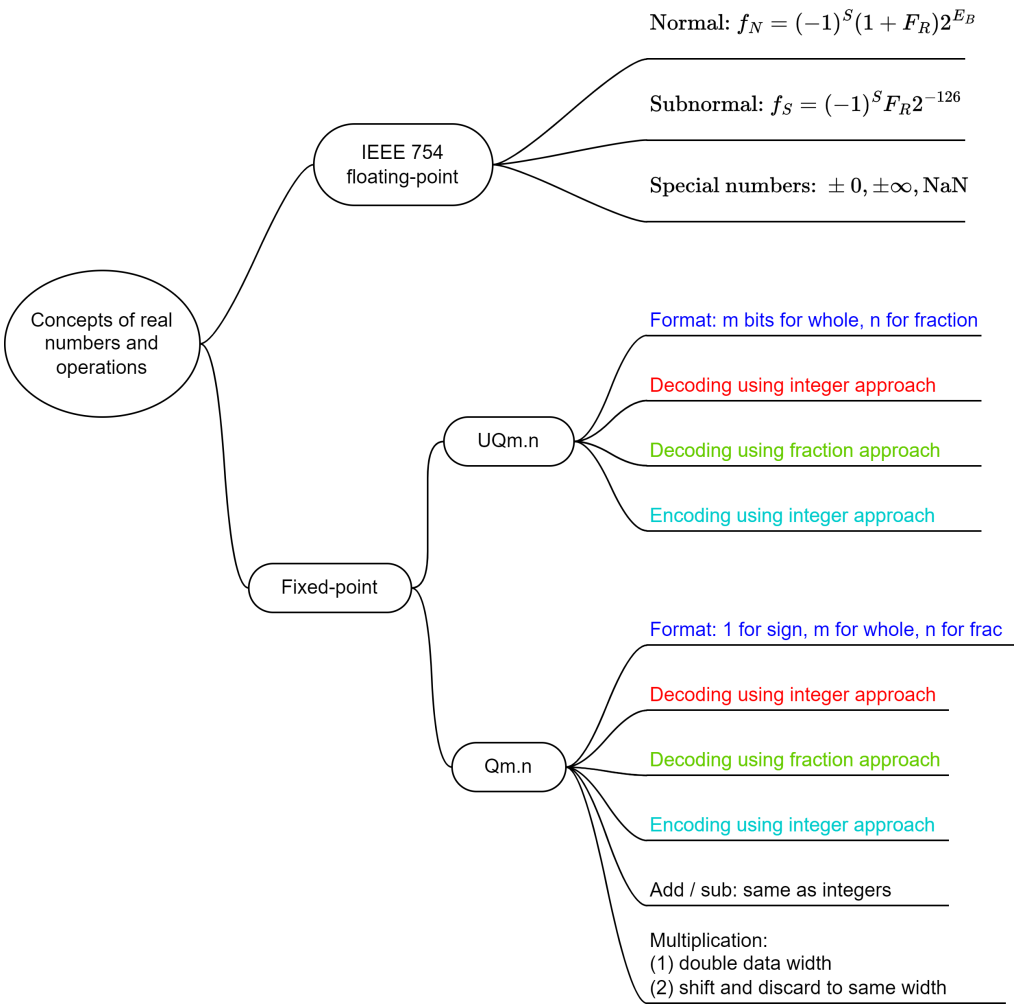

Figure 5: A concept map of the main topics of this lecture.

13.6.2 Including DSP library in a project in CubeIDE

13.6.3 Using FPU and floating-point functions